

Perl Data Structures - Complete Practice Test

Instructions

- Try to solve each problem on your own first
 - Answers are at the bottom
 - The test progresses from basic to advanced
 - Each section builds on previous concepts
-

PART 1: Arrays of Arrays (AoA)

Question 1: Create an Array of Arrays

Create an array called `@matrix` that represents this 3x3 grid:

```
1 2 3
4 5 6
7 8 9
```

Question 2: Access Single Elements

Given:

```
perl

my @AoA = (
    [ "apple", "banana", "cherry" ],
    [ "dog", "elephant", "fox" ],
    [ "guitar", "harp", "piano" ]
);
```

Write code to:

- a) Print "elephant"
- b) Print "cherry"
- c) Print "piano"

Question 3: Add to Existing Arrays

Using the `@AoA` from Question 2:

- a) Add "dragonfruit" to the end of the first row

- b) Add a new fourth row: `[ "jazz", "rock", "blues" ]`

Question 4: Loop Through All Elements

Using the `@AoA` from Question 2, write code to:

- a) Print each row on a separate line
- b) Print each element with its coordinates (row, column)

Question 5: Common Mistake - Fix This Code

What's wrong with this code? Fix it.

```
perl

my @data;
my @row;
for my $i (1..3) {
    @row = ($i, $i*2, $i*3);
    $data[$i] = \@row; # What's the problem here?
}
```

PART 2: Hashes of Arrays (HoA)

Question 6: Create a Hash of Arrays

Create a hash called `%classes` where:

- "Math" has students: "Alice", "Bob", "Charlie"
- "English" has students: "David", "Eve"
- "Science" has students: "Frank", "Grace", "Henry", "Iris"

Question 7: Access Elements

Using the `%classes` from Question 6:

- a) Print all students in the Math class
- b) Print the first student in the Science class
- c) Print the number of students in English

Question 8: Add and Modify

Using the `%classes` from Question 6:

- a) Add "Jack" to the Math class
- b) Add a new class "History" with students "Kate" and "Leo"
- c) Change the second student in English to "Emma"

Question 9: Loop Through Hash of Arrays

Using the `%classes` from Question 6:

- a) Print each class name with its students
- b) Print only classes that have more than 2 students
- c) Count total number of students across all classes

Question 10: Sort by Array Size

Using the `%classes` from Question 6, print all classes sorted by number of students (largest first).

PART 3: Arrays of Hashes (AoH)

Question 11: Create an Array of Hashes

Create an array `@employees` with these records:

- Name: "John", Age: 30, Department: "Sales"
- Name: "Sarah", Age: 28, Department: "IT"
- Name: "Mike", Age: 35, Department: "Sales"

Question 12: Access Hash Fields

Using `@employees` from Question 11:

- a) Print Sarah's department
- b) Print the age of the first employee
- c) Print all information about Mike

Question 13: Add and Modify Records

Using `@employees` from Question 11:

- a) Add a new employee: Name "Emma", Age 26, Department "HR"
- b) Change John's age to 31
- c) Add a "salary" field to Sarah's record with value 75000

Question 14: Loop and Filter

Using `@employees` from Question 11:

- a) Print names of all employees
- b) Print only employees in the Sales department
- c) Print employees older than 29

Question 15: Calculate from Records

Using `@employees` from Question 11:

- a) Find the average age of all employees
 - b) Count how many employees are in each department
-

PART 4: Hashes of Hashes (HoH)

Question 16: Create a Hash of Hashes

Create `%inventory` representing store inventory:

- "Electronics": { laptops => 15, phones => 30, tablets => 20 }
- "Clothing": { shirts => 50, pants => 40, shoes => 25 }
- "Food": { apples => 100, bread => 45, milk => 60 }

Question 17: Access Nested Values

Using `%inventory` from Question 16:

- a) Print the number of laptops
- b) Print the number of apples
- c) Print all items in the Clothing department

Question 18: Add and Update

Using `%inventory` from Question 16:

- a) Add "keyboards" with quantity 12 to Electronics
- b) Update the number of phones to 35
- c) Add a new department "Books" with items: novels => 80, textbooks => 30

Question 19: Loop Through Hash of Hashes

Using `%inventory` from Question 16:

- a) Print each department with all its items
- b) Print total number of items in each department
- c) Find which department has the most total items

Question 20: Complex Queries

Using `%inventory` from Question 16:

- a) Print all items across all departments that have quantity > 40
 - b) Calculate total inventory (sum of all quantities)
 - c) Find which item has the highest quantity
-

PART 5: Mixed and Complex Structures

Question 21: Nested Structure

Create this structure:

```
perl

%library = (
  "Fiction" => [
    { title => "1984", author => "Orwell", year => 1949 },
    { title => "Dune", author => "Herbert", year => 1965 }
  ],
  "Non-Fiction" => [
    { title => "Sapiens", author => "Harari", year => 2011 }
  ]
);
```

- a) Print the title of the first Fiction book
- b) Print the author of "Sapiens"
- c) Add a new Fiction book: "Brave New World" by "Huxley", 1932

Question 22: Deep Nesting

Given:

```
perl
```

```

my %company = (
  Sales => {
    manager => "Alice",
    employees => [
      { name => "Bob", salary => 50000 },
      { name => "Carol", salary => 55000 }
    ]
  },
  IT => {
    manager => "Dave",
    employees => [
      { name => "Eve", salary => 70000 },
      { name => "Frank", salary => 75000 }
    ]
  }
);

```

- a) Print the Sales manager's name
- b) Print Carol's salary
- c) Print all employee names in the IT department
- d) Give Eve a 10% raise

Question 23: Build From Scratch

Create a structure to represent this data:

- Three courses: "Math101", "Eng101", "Sci101"
- Each course has: instructor name, room number, and list of enrolled students
- Add at least 2 students to each course

Question 24: Reading Data

Write code that reads this input format and builds a hash of arrays:

```

Team1: Alice Bob Charlie
Team2: Dave Eve
Team3: Frank Grace Henry Iris

```

(Assume the data comes from a file or STDIN)

Question 25: Transformation

Given an array of hashes (employees with name and department), transform it into a hash of arrays where keys are departments and values are arrays of employee names.

PART 6: Modern Syntax and Best Practices

Question 26: Postfix Dereferencing

Rewrite using modern postfix syntax (`->@*` and `->%*`):

```
perl

push @{$AoA[0]}, "new item";
my @keys = keys %{$HoH{family}};
my $count = scalar @{$HoA{team}};
```

Question 27: Reference Construction

Which of these is the SAFEST way to build an array of arrays in a loop, and why?

```
perl

# Option A
for my $i (1..5) {
    my @temp = get_data($i);
    $AoA[$i] = \@temp;
}

# Option B
for my $i (1..5) {
    my @temp = get_data($i);
    $AoA[$i] = [ @temp ];
}

# Option C
my @temp;
for my $i (1..5) {
    @temp = get_data($i);
    $AoA[$i] = \@temp;
}
```

Question 28: Debugging

What will this print and why?

```
perl
```

```
my @data = (  
    "a", "b",  
    "c", "d"  
);  
print @data;
```

Question 29: Common Pitfall

Fix this code that's trying to create a hash of arrays from user input:

```
perl  
  
my %teams;  
while (my $line = <DATA>) {  
    chomp $line;  
    my ($team, $member) = split /:/, $line;  
    push @{$teams{$team}}, $member; # Is this correct?  
}  
  
__DATA__  
Red:Alice  
Red:Bob  
Blue:Charlie
```

Question 30: Deep Copy vs Reference

Explain the difference:

```
perl  
  
my @original = ([1, 2], [3, 4]);  
  
# Method 1  
my @copy1 = @original;  
$copy1[0][0] = 99;  
  
# Method 2  
my @copy2;  
for my $row (@original) {  
    push @copy2, [ @$row ];  
}  
$copy2[0][0] = 88;  
  
# What does $original[0][0] contain now?
```


BONUS CHALLENGES

Challenge 1: Student Grade System

Create a complete student grade management system with:

- Hash of hashes: students → subjects → grades
- Functions to: add grade, calculate average, find top student

Challenge 2: Social Network

Create a simple social network structure:

- Hash of hashes: users → friends → friend_data
- Each friend has: name, since (year), mutual_friends count
- Function to find mutual friends between two users

Challenge 3: Inventory Management

Build a complete inventory system:

- Multi-level structure with categories, items, warehouses
 - Each item has: name, quantity per warehouse, price, supplier info
 - Functions to: check total stock, transfer between warehouses, low stock alert
-

ANSWERS

Part 1: Arrays of Arrays

Answer 1:

```
perl

my @matrix = (
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
);
```

Answer 2:

```
perl
```

```
# a) Print "elephant"
```

```
print $AoA[1][1];
```

```
# b) Print "cherry"
```

```
print $AoA[0][2];
```

```
# c) Print "piano"
```

```
print $AoA[2][2];
```

Answer 3:

```
perl
```

```
# a) Add "dragonfruit" to first row
```

```
push @{$AoA[0]}, "dragonfruit";
```

```
# or modern: push $AoA[0]->@*, "dragonfruit";
```

```
# b) Add new fourth row
```

```
push @AoA, [ "jazz", "rock", "blues" ];
```

Answer 4:

```
perl
```

```
# a) Print each row
```

```
for my $row ( @AoA ) {
```

```
    print "$row\n";
```

```
}
```

```
# or: print "$row->@*\n";
```

```
# b) Print with coordinates
```

```
for my $i (0..$#AoA) {
```

```
    for my $j (0..${$AoA[$i]}) {
```

```
        print "($i,$j) = $AoA[$i][$j]\n";
```

```
    }
```

```
}
```

Answer 5:

```
perl
```

PROBLEM: @row is reused, all references point to same array

FIX: Use array constructor []

```
my @data;
for my $i (1..3) {
    my @row = ($i, $i*2, $i*3);
    $data[$i] = [ @row ]; # Create new array each time
}
```

Part 2: Hashes of Arrays

Answer 6:

```
perl

my %classes = (
    Math => [ "Alice", "Bob", "Charlie" ],
    English => [ "David", "Eve" ],
    Science => [ "Frank", "Grace", "Henry", "Iris" ]
);
```

Answer 7:

```
perl

# a) All students in Math
print "@{$classes{Math}}\n";

# b) First student in Science
print $classes{Science}[0];

# c) Number of students in English
print scalar @{$classes{English}};
# or: print scalar $classes{English}->@*;
```

Answer 8:

```
perl
```

```
# a) Add Jack to Math
push @{$classes}{Math}, "Jack";

# b) Add History class
$classes{History} = [ "Kate", "Leo" ];

# c) Change second student in English
$classes{English}[1] = "Emma";
```

Answer 9:

```
perl

# a) Print each class with students
for my $class (keys %classes) {
    print "Class: @{$classes{$class}}\n";
}

# b) Classes with more than 2 students
for my $class (keys %classes) {
    if (@{$classes{$class}} > 2) {
        print "Class: @{$classes{$class}}\n";
    }
}

# c) Total students
my $total = 0;
for my $class (keys %classes) {
    $total += @{$classes{$class}};
}
print "Total: $total\n";
```

Answer 10:

```
perl

for my $class (sort { @{$classes{$b}} <=> @{$classes{$a}} } keys %classes) {
    print "Class ( " . scalar(@{$classes{$class}}) . " students)\n";
}
```

Part 3: Arrays of Hashes

Answer 11:

```
perl

my @employees = (
    { Name => "John", Age => 30, Department => "Sales" },
    { Name => "Sarah", Age => 28, Department => "IT" },
    { Name => "Mike", Age => 35, Department => "Sales" }
);
```

Answer 12:

```
perl

# a) Sarah's department
print $employees[1]{Department};

# b) Age of first employee
print $employees[0]{Age};

# c) All info about Mike
for my $key (keys %{ $employees[2] }) {
    print "$key: $employees[2]{$key}\n";
}
```

Answer 13:

```
perl

# a) Add Emma
push @employees, { Name => "Emma", Age => 26, Department => "HR" };

# b) Change John's age
$employees[0]{Age} = 31;

# c) Add salary to Sarah
$employees[1]{salary} = 75000;
```

Answer 14:

```
perl
```

a) All names

```
for my $emp (@employees) {  
    print "$emp->{Name}\n";  
}
```

b) Sales employees

```
for my $emp (@employees) {  
    if ($emp->{Department} eq "Sales") {  
        print "$emp->{Name}\n";  
    }  
}
```

c) Employees older than 29

```
for my $emp (@employees) {  
    if ($emp->{Age} > 29) {  
        print "$emp->{Name} ($emp->{Age})\n";  
    }  
}
```

Answer 15:

perl

a) Average age

```
my $sum = 0;  
$sum += $_->{Age} for @employees;  
print "Average: ", $sum / @employees, "\n";
```

b) Count by department

```
my %dept_count;  
for my $emp (@employees) {  
    $dept_count{$emp->{Department}}++;  
}  
for my $dept (keys %dept_count) {  
    print "$dept: $dept_count{$dept}\n";  
}
```

Part 4: Hashes of Hashes

Answer 16:

perl

```
my %inventory = (  
    Electronics => { laptops => 15, phones => 30, tablets => 20 },  
    Clothing    => { shirts => 50, pants => 40, shoes => 25 },  
    Food        => { apples => 100, bread => 45, milk => 60 }  
);
```

Answer 17:

```
perl  
  
# a) Number of laptops  
print $inventory{Electronics}{laptops};  
  
# b) Number of apples  
print $inventory{Food}{apples};  
  
# c) All Clothing items  
for my $item (keys %{$inventory{Clothing}}) {  
    print "$item: $inventory{Clothing}{$item}\n";  
}
```

Answer 18:

```
perl  
  
# a) Add keyboards  
$inventory{Electronics}{keyboards} = 12;  
  
# b) Update phones  
$inventory{Electronics}{phones} = 35;  
  
# c) Add Books department  
$inventory{Books} = { novels => 80, textbooks => 30 };
```

Answer 19:

```
perl
```

a) Print all departments and items

```
for my $dept (keys %inventory) {  
    print "$dept:\n";  
    for my $item (keys %{ $inventory{$dept} }) {  
        print "  $item: $inventory{$dept}{$item}\n";  
    }  
}
```

b) Total items per department

```
for my $dept (keys %inventory) {  
    my $total = 0;  
    $total += $_ for values %{ $inventory{$dept} };  
    print "$dept: $total items\n";  
}
```

c) Department with most items

```
my %dept_totals;  
for my $dept (keys %inventory) {  
    $dept_totals{$dept} += $_ for values %{ $inventory{$dept} };  
}  
my ($max_dept) = sort { $dept_totals{$b} <=> $dept_totals{$a} } keys %dept_totals;  
print "Most items in: $max_dept\n";
```

Answer 20:

perl


```
# a) Items with quantity > 40
for my $dept (keys %inventory) {
    for my $item (keys %{ $inventory{$dept}}) {
        if ( $inventory{$dept}{$item} > 40) {
            print "$dept - $item: $inventory{$dept}{$item}\n";
        }
    }
}
```

```
# b) Total inventory
my $grand_total = 0;
for my $dept (keys %inventory) {
    $grand_total += $_ for values %{ $inventory{$dept}};
}
print "Total inventory: $grand_total\n";
```

```
# c) Highest quantity item
my ($max_item, $max_dept, $max_qty) = ("", "", 0);
for my $dept (keys %inventory) {
    for my $item (keys %{ $inventory{$dept}}) {
        if ( $inventory{$dept}{$item} > $max_qty) {
            $max_qty = $inventory{$dept}{$item};
            $max_item = $item;
            $max_dept = $dept;
        }
    }
}
print "Highest: $max_item in $max_dept ($max_qty)\n";
```

Part 5: Mixed and Complex Structures

Answer 21:

```
perl
```

```
my %library = (
    Fiction => [
        { title => "1984", author => "Orwell", year => 1949 },
        { title => "Dune", author => "Herbert", year => 1965 }
    ],
    "Non-Fiction" => [
        { title => "Sapiens", author => "Harari", year => 2011 }
    ]
);
```

a) First Fiction book title

```
print $library{Fiction}[0]{title};
```

b) Sapiens author

```
print $library{"Non-Fiction"}[0]{author};
```

c) Add new Fiction book

```
push @{$library{Fiction}}, {
    title => "Brave New World",
    author => "Huxley",
    year => 1932
};
```

Answer 22:

```
perl
```

a) Sales manager

```
print $company{Sales}{manager};
```

b) Carol's salary

```
print $company{Sales}{employees}[1]{salary};
```

c) IT employee names

```
for my $emp (@{$company{IT}{employees}}) {
    print "$emp->{name}\n";
}
```

d) Give Eve a raise

```
$company{IT}{employees}[0]{salary} *= 1.10;
```

Answer 23:

```
perl
```

```

my %courses = (
    Math101 => {
        instructor => "Dr. Smith",
        room => "101A",
        students => [ "Alice", "Bob", "Charlie" ]
    },
    Eng101 => {
        instructor => "Prof. Jones",
        room => "205B",
        students => [ "Dave", "Eve" ]
    },
    Sci101 => {
        instructor => "Dr. Brown",
        room => "Lab3",
        students => [ "Frank", "Grace", "Henry" ]
    }
);

```

Answer 24:

```

perl

my %teams;
while (<DATA>) {
    chomp;
    my ($team, $members) = split /\s*/, $_, 2;
    $teams{$team} = [ split /\s+/, $members ];
}

__DATA__
Team1: Alice Bob Charlie
Team2: Dave Eve
Team3: Frank Grace Henry Iris

```

Answer 25:

```

perl

```

```

my @employees = (
    { name => "Alice", dept => "Sales" },
    { name => "Bob", dept => "IT" },
    { name => "Carol", dept => "Sales" },
    { name => "Dave", dept => "IT" }
);

my %by_dept;
for my $emp (@employees) {
    push @{$by_dept{$emp->{dept}}}, $emp->{name};
}

```

Part 6: Modern Syntax and Best Practices

Answer 26:

```

perl

# Modern postfix syntax
push $AoA[0]->@*, "new item";
my @keys = keys $HoH{family}->%*;
my $count = scalar $HoA{team}->@*;

```

Answer 27:

```

perl

# Answer: Option B is SAFEST
#
# Option A: Safe because 'my @temp' creates new array each iteration
# Option B: SAFEST - explicitly creates new array with []
# Option C: DANGEROUS - all references point to same @temp
#
# Best practice: Use Option B with [] constructor

```

Answer 28:

```

perl

# It prints: ARRAY(0x...)ARRAY(0x...)
# Because @data contains references, not the actual arrays
# To print properly: print "@{$_}\n" for @data;

```

Answer 29:

```
perl

# The code is actually CORRECT!
# The autovivification feature creates the array automatically
# But for clarity, you could initialize:

my %teams;
while (my $line = <DATA>) {
    chomp $line;
    my ($team, $member) = split /:/, $line;
    $teams{$team} //= []; # Initialize if needed (optional)
    push @{$teams{$team}}, $member;
}
```

Answer 30:

```
perl

# After both operations, $original[0][0] contains 99
#
# Method 1 (shallow copy): @copy1 and @original share references
# Changing $copy1[0][0] also changes $original[0][0]
#
# Method 2 (deep copy): Creates new arrays
# $copy2[0][0] change doesn't affect $original
#
# $original[0][0] = 99 (from Method 1)
```

Bonus Challenge Solutions

Challenge 1: Student Grade System

```
perl
```

```

my %grades = (
    Alice => { Math => 85, English => 90, Science => 88 },
    Bob  => { Math => 92, English => 85, Science => 90 },
    Carol => { Math => 78, English => 95, Science => 82 }
);

sub add_grade {
    my ($student, $subject, $grade) = @_;
    $grades{$student}{$subject} = $grade;
}

sub calculate_average {
    my ($student) = @_;
    my $sum = 0;
    my $count = 0;
    for my $grade (values %{ $grades{$student} }) {
        $sum += $grade;
        $count++;
    }
    return $count ? $sum / $count : 0;
}

sub find_top_student {
    my %averages;
    for my $student (keys %grades) {
        $averages{$student} = calculate_average($student);
    }
    my ($top) = sort { $averages{$b} <=> $averages{$a} } keys %averages;
    return $top;
}

# Usage
add_grade("Alice", "History", 93);
print "Alice average: ", calculate_average("Alice"), "\n";
print "Top student: ", find_top_student(), "\n";

```

Challenge 2: Social Network

```
perl
```

```
my %network = (  
  Alice => {  
    Bob => { since => 2020, mutual => 3 },  
    Carol => { since => 2019, mutual => 5 }  
  },  
  Bob => {  
    Alice => { since => 2020, mutual => 3 },  
    Dave => { since => 2021, mutual => 2 }  
  }  
);  
  
sub find_mutual_friends {  
  my ($user1, $user2) = @_;  
  my %friends1 = map { $_ => 1 } keys %{$network{$user1}};  
  my %friends2 = map { $_ => 1 } keys %{$network{$user2}};  
  
  my @mutual;  
  for my $friend (keys %friends1) {  
    push @mutual, $friend if $friends2{$friend};  
  }  
  return @mutual;  
}
```

Challenge 3: Inventory Management

perl

```

my %inventory = (
    Electronics => {
        Laptop => {
            warehouses => { WH1 => 10, WH2 => 15 },
            price => 999,
            supplier => "TechCorp"
        },
        Mouse => {
            warehouses => { WH1 => 50, WH2 => 30 },
            price => 25,
            supplier => "TechCorp"
        }
    }
);

```

```

sub total_stock {
    my ($category, $item) = @_;
    my $total = 0;
    for my $qty (values %{ $inventory{ $category }{ $item }{ warehouses } }) {
        $total += $qty;
    }
    return $total;
}

```

```

sub transfer {
    my ($category, $item, $from, $to, $qty) = @_;
    if ( $inventory{ $category }{ $item }{ warehouses }{ $from } >= $qty ) {
        $inventory{ $category }{ $item }{ warehouses }{ $from } -= $qty;
        $inventory{ $category }{ $item }{ warehouses }{ $to } += $qty;
        return 1;
    }
    return 0;
}

```

```

sub low_stock_alert {
    my ($threshold) = @_;
    for my $cat (keys %inventory) {
        for my $item (keys %{ $inventory{ $cat } }) {
            my $total = total_stock($cat, $item);
            if ($total < $threshold) {
                print "LOW STOCK: $cat - $item ($total units)\n";
            }
        }
    }
}

```

Self-Check Questions

After completing this test, you should be able to answer:

1. ✓ What's the difference between `[]` and `\@array`?
2. ✓ When should I use postfix dereferencing (`->@*`)?
3. ✓ Why does my code print `ARRAY(0x...)`?
4. ✓ How do I add elements to nested structures?
5. ✓ How do I loop through multi-level data?
6. ✓ What's the most common mistake when building AoA?
7. ✓ How do I safely copy nested structures?
8. ✓ When does Perl auto-create (autovivify) references?
9. ✓ How do I debug complex data structures?
10. ✓ What's the best way to initialize nested structures?

Congratulations! If you can solve these problems, you have a solid understanding of Perl data structures!